UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

COURSE CODE: CSE 402

COURSE TITLE: Operating System Lab

EXPERIMENT: Lab 02 - SJF | Comparison with FCFS

Submitted by:

Jannatul Ferdoushi Islam
Student ID: 23101119
Section: C-1
Semester: 4-1

Submitted to:

Atia Rahman Orthi,
Lecturer,
University of Asia Pacific

Lab Report: SJF Comparison with FCFS

Problem Statement

Develop and execute the SJF (Non-Preemptive) and FCFS CPU scheduling methods for the supplied processes. Determine Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT, and Average WT, then compare the results of the two scheduling approaches.

Objective

The purpose of this experiment is to learn how Non-Preemptive Shortest Job First scheduling works and to evaluate its performance against First Come First Serve scheduling. The experiment also demonstrates how a change in process execution order influences CT, TAT, WT, and their average values.

Given Data

Process ID	Arrival Time (AT)	Burst Time (BT)
P1	2	5
P2	3	1
P3	0	2
P4	1	3
P5	1	6

Working Principle

SJF (Non-Preemptive): The CPU chooses the process having the smallest Burst Time from all processes that have already arrived. After a process begins, it runs until it finishes. The scheduler then checks the ready processes again and selects the shortest available job.

FCFS: Processes are handled according to arrival order. The earliest arriving process receives the CPU first. When no process is ready, the CPU remains idle until the next process arrives.

For both methods, **TAT = CT - AT** and **WT = TAT - BT**. Although the formulas remain unchanged, the final values can vary because each algorithm may execute the processes in a different sequence.

Why SJF Can Perform Better

SJF favors shorter ready jobs, allowing several small processes to complete sooner. This commonly lowers the overall average waiting time and turnaround time. For the data used in this lab, SJF gives smaller Average TAT and Average WT than FCFS.

Limitations of SJF

Starvation: A process with a high Burst Time can remain waiting for a long period when shorter jobs continue to become available.
Burst Time Estimation: SJF depends on knowing or predicting CPU burst duration, which may not be exact in practical systems.
Fairness: Since short jobs receive preference, longer jobs may experience less favorable service.

FCFS and Starvation

FCFS generally prevents starvation because processes are served in the same order in which they arrive, so each process eventually receives CPU time. However, FCFS may create a convoy effect when one

long process at the front causes several short processes to wait.

Where FCFS and SJF Are Used

FCFS is suitable for straightforward queues such as printer jobs, simple batch systems, ticket or service queues, and cases where arrival-order fairness matters. SJF is more suitable for batch or job scheduling when execution time can be estimated and minimizing average waiting time is a major goal.

Source Code

```
process = ["P1", "P2", "P3", "P4", "P5"]
at = [2, 3, 0, 1, 1]
bt = [5, 1, 2, 3, 6]

n = len(process)
ct = [0] * n
tat = [0] * n
wt = [0] * n
done = [0] * n

time = 0
completed = 0

while completed < n:
    x = -1
    for i in range(n):
        if at[i] <= time and done[i] == 0:
            if x == -1 or bt[i] < bt[x]:
                x = i

    if x == -1:
        time = time + 1
    else:
        time = time + bt[x]
        ct[x] = time
        done[x] = 1
        completed = completed + 1

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("SJF RESULT")
print(f"{'Process':<10}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")
for i in range(n):
    print(f"{process[i]:<10}{at[i]:<6}{bt[i]:<6}{ct[i]:<6}{tat[i]:<6}{wt[i]:<6}")

sjf_avg_tat = sum(tat) / n
sjf_avg_wt = sum(wt) / n

print()
print("Average TAT =", sjf_avg_tat)
print("Average WT =", sjf_avg_wt)
print()
print()

# FCFS
processes = [
    ["P1", 2, 5],
    ["P2", 3, 1],
    ["P3", 0, 2],
    ["P4", 1, 3],
    ["P5", 1, 6]
]

processes.sort(key=lambda x: x[1])
time = 0
total_tat = 0
total_wt = 0

print("FCFS RESULT")
print(f"{'Process':<10}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")

for p in processes:
    pid = p[0]
    arrival = p[1]
```

```
    burst = p[2]

    if time < arrival:
        time = arrival

    time = time + burst
    completion = time
    turnaround = completion - arrival
    waiting = turnaround - burst

    total_tat = total_tat + turnaround
    total_wt = total_wt + waiting

    print(f"{pid:<10}{arrival:<6}{burst:<6}{completion:<6}{turnaround:<6}{waiting:<6}")

fcfs_avg_tat = total_tat / len(processes)
fcfs_avg_wt = total_wt / len(processes)

print()
print("Average TAT =", fcfs_avg_tat)
print("Average WT =", fcfs_avg_wt)
print()
print()

if sjf_avg_tat < fcfs_avg_tat:
    print("SJF is better TAT than FCFS")
else:
    print("FCFS is better TAT than SJF")

if sjf_avg_wt < fcfs_avg_wt:
    print("SJF is better WT than FCFS")
else:
    print("FCFS is better WT than SJF")
```

Output

```
SJF RESULT
Process    AT    BT    CT    TAT    WT
P1         2     5    11     9     4
P2         3     1     6     3     2
P3         0     2     2     2     0
P4         1     3     5     4     1
P5         1     6    17    16    10

Average TAT = 6.8
Average WT = 3.4

FCFS RESULT
Process    AT    BT    CT    TAT    WT
P3         0     2     2     2     0
P4         1     3     5     4     1
P5         1     6    11    10     4
P1         2     5    16    14     9
P2         3     1    17    14    13

Average TAT = 8.8
Average WT = 5.4

SJF is better TAT than FCFS
SJF is better WT than FCFS
```

Comparison between SJF and FCFS

Point	**SJF**	**FCFS**
Process Selection	Chooses shortest ready job	Chooses by arrival order
Average TAT	6.8	8.8
Average WT	3.4	5.4
Starvation	Long jobs may starve	Usually does not occur
Convoy Effect	Generally less common	Can occur

Point	SJF	FCFS
Burst Time Information	Must be known/estimated	Not necessary
Fairness	Favors shorter processes	Fair by arrival sequence
Implementation	More scheduling decisions	Easy and straightforward
Performance here	Better	Lower than SJF

Discussion

Both Non-Preemptive SJF and FCFS were tested with the same five processes. Their main difference is the rule used to choose the next process. SJF examines the processes that are already ready and runs the one with the smallest Burst Time, whereas FCFS simply follows arrival order. Because of this difference, the processes finish at different times, which also changes their turnaround and waiting times.

For this experiment, SJF achieved an Average TAT of 6.8 and an Average WT of 3.4. FCFS resulted in an Average TAT of 8.8 and an Average WT of 5.4. Therefore, SJF is the more efficient algorithm for this particular dataset because both measured averages are lower.

The result does not mean SJF is always the best choice. It can delay long processes for a considerable time and requires the Burst Time to be available or predicted. FCFS is easier to implement and provides service according to arrival order, but a long process can hold up shorter processes and create the convoy effect.

Conclusion

This lab successfully implemented and evaluated SJF Non-Preemptive and FCFS CPU scheduling. CT, TAT, WT, Average TAT, and Average WT were obtained for both methods. With the given process set, SJF produced the better result because its Average TAT and Average WT were lower. The comparison also highlights the trade-off between SJF's efficiency in reducing average waiting and FCFS's simplicity and arrival-order fairness.

GitLink:

https://github.com/nuzaiba119/OS_LAB/tree/main/LAB2